Technical Document

Manage Backlog — SRS

SOFTWARE REQUIREMENTS SPECIFICATION

Manage Backlog · Sprint Planning Board

Software Requirements Specification (SRS) — Manage Backlog

Feature: Manage Backlog (Sprint planning board) **Apex controller:** `force-app/main/default/classes/controller/managebacklog/ManageBacklogController.cls` **LWC:** `force-app/main/default/lwc/manageBacklog/` (parent) with children `c-ao-ticket-item`, `c-ao-create-ticket-modal`, `c-ticket-view`, `c-choose-project`, and the `ao-*` design-system controls. **Date:** 2026-06-20

1. Purpose & Scope

The Manage Backlog page is the sprint-planning surface of the Jira Clone. For a single selected **Project** it lets a user:

- See one **Backlog** container and **every non-completed, non-deleted Sprint** container.
- Create / edit / start / complete / delete sprints.
- Create tickets directly into the backlog or into a sprint (one reusable Create-Ticket modal).
- Move tickets backlog→sprint, sprint→backlog, and reorder within the same container — with the

sprint story-point totals kept in sync on every move.

- Operate on a ticket inline (delete, edit summary, change status, change priority, change assignee, set/clear epic, add & expand sub-tasks) and open a right-side ticket peek panel.
- Page through tickets in every container (page size = **5**).

The project is selected once and persisted in `localStorage('projectId')`; if absent the page shows the **Choose Project** splash first.

2. Actors

Actor	Description
Project Member (User)	The authenticated Salesforce user, who must be an *active* `ProjectMember__c` of the project. Performs all backlog/sprint/ticket actions. The current user is recorded as ticket `Creator__c` on create.
System	Apex services that auto-number sprints/tickets, recompute sprint story points, assign ticket ordering scores, and enforce workflow transitions.

3. Functional Requirements

3.1 Project Selection and Data Loading

- **FR-1** The system shall allow a user to select a project before accessing backlog management functions.
- **FR-2** The system shall load all backlog management data associated with the selected project, including sprints, tickets, statuses, ticket types, project members, priorities, and epics .
- **FR-3** The system shall display only sprints whose lifecycle state is neither Completed nor Deleted.

3.2 Backlog Management

- **FR-4** The system shall provide access to all active backlog tickets that are not assigned to a sprint, they must be placed in order.
- **FR-5** The system shall allow users to initiate sprint creation and ticket creation from the backlog context.

3.3 Sprint Management

- **FR-6** The System must show for each sprint: name, `StartDate → endDate`, status badge, and action buttons **Start** (hidden when complete), **Complete**, **Edit**, **Delete**, **+ Ticket**.
- **FR-7** The system shall provide access to sprint goals, sprint progress, and sprint tickets.
- **FR-8** Sprint tickets are paginated 5 per page with Prev/Next + "Page N".
- **FR-9** The system shall allow users to start a sprint that satisfies sprint lifecycle rules.
- **FR-10** The system shall allow users to complete an active sprint
- **FR-11** The system shall allow users to modify sprint attributes, including duration, start date, and goal.
- **FR-12 (Delete)** Clicking **Delete** shows a confirmation pop-up; on confirm the sprint is soft-deleted and its tickets are moved back to the backlog.
- **FR-13 (+ Ticket)** Opens the reusable Create-Ticket modal in *sprint* mode (ticket is created already linked to that sprint).

3.4 Ticket Creation

- **FR-14** The system shall allow users to create tickets using summary, description, story points, ticket type, status, and priority information.

- **FR-15** The system shall associate newly created tickets with either the backlog or a sprint based on the creation context and shall update sprint metrics (Total Story Points – Ended Story Points) when applicable.

3.5 Ticket Management

- **FR-16** The system shall provide access to ticket information including identifier, summary, status, priority, assignee, epic association, type, and story points.
- **FR-17** The system shall allow users to update ticket summary, status, priority, assignee, epic association, and subtask information, and shall allow ticket deletion.
- **FR-18** The system shall allow users to associate a ticket with an existing epic or create a new epic and associate it with the ticket.
- **FR-19** The system shall allow users to select multiple tickets and perform bulk deletion.

3.6 Subtask Management

- **FR-20** The system shall allow users to create subtasks associated with a parent ticket.
- **FR-21** The system shall allow users to view, update, and delete subtasks associated with a ticket.

3.7 Ticket Movement and Ordering

- **FR-22** The system shall allow users to move backlog tickets to a sprint and shall update sprint metrics accordingly.
- **FR-23** The system shall allow users to remove tickets from a sprint and return them to the backlog while updating sprint metrics accordingly.
- **FR-24** The system shall allow users to reorder tickets within the same scope and shall maintain the resulting ordering.

3.8 Ticket Details and Relationships

- **FR-25** The system shall provide access to detailed ticket information and editing capabilities.
- **FR-26** The system shall allow users to open and close a detailed ticket view**.**
- **FR-27** The system shall allow users to modify ticket summary information from the detailed ticket view**.**
- **FR-28** he system shall allow users to modify ticket status from the detailed ticket view subject based on the workflow Transition + validate field rule**.**
- **FR-29** The system shall allow users to modify ticket descriptions using rich-text content**.**

- **FR-30** The system shall allow users to create, view, and manage relationships between tickets**.**
- **FR-31** The system shall allow users to create and manage subtasks from the detailed ticket view**.**

4. Validation Rules

4.1 Client-side (LWC validators)

Sprint form — `backlogSprintValidator.js`

- **VR-1** Duration is **required**.
- **VR-2** Duration must be a **whole number between 1 and 999** (`Sprint__c.Duration__c` precision 3, scale 0).
- **VR-3** Goal is **required** (non-blank after trim).
- **VR-4** Start Date is marked required in the modal UI (`required` on the input). *(The JS validator itself enforces only duration + goal; Name is auto-generated server-side.)*

Ticket form / inline — `backlogTicketValidator.js`

- **VR-5** Name **required**, **≤ 80** chars (`Ticket__c` name field).
- **VR-6** Summary **required**, **≤ 255** chars (`Summary__c`).
- **VR-7** Ticket Type **required** (for create).
- **VR-8** Current State **required**.
- **VR-9** Story Point optional; if provided must be a **whole number 0–99** (`StoryPoint__c` precision 2, scale 0).
- **VR-10** Priority on inline update **required** and must be one of **Critical / High / Medium / Low**.

4.2 Server-side (controller input guards)

- **VR-11** Every `@AuraEnabled` method blank-checks its required parameters and returns `APIResponse(false, '<field> is required')` before doing any work (e.g. `projectId`, `ticketId`, `sprintId`, `summary`, `ticketTypeId`, `searchTerm`, `priority`, `memberId`, `fromStatusId`/`toStatusId`).

- **VR-12** Referenced records must exist / be valid via `DomainCorrectness.require*` (project, sprint, ticket, ticket type, status, epic, workflow, member). Optional references use `requireOptional*`.

- **VR-13** `loadTicketBySearchTerm` escapes all SOSL/SOQL special characters in the search term

before building the query.

5. Business Rules

5.1 Sprint lifecycle

- **BR-1** A new sprint is created with `RecordStatus__c = 'future'`, `TotalStoryPoint__c = 0`, `TotalEndedStoryPoint__c = 0`, `MaxScore__c = '000000'`, and an auto-generated `Name` (`NameSequenceService.nextSprintName`).
- **BR-2 A sprint Name must be unique per project** among non-deleted sprints (`DomainCompleteValidator.requireSprintNameUniquePerProject`).
- **BR-3** Sprint status flow is **future → in_progress → completed**, plus a soft **deleted** state.
- **BR-4 Start** requires the sprint to be startable and that **no other sprint in the project is already `in_progress`** (`requireNoActiveSprintInProject`) — at most one active sprint per project.
- **BR-5 Complete** requires the sprint to be completable; a completed sprint is excluded from `loadUncompleteSprintsByProject`, so it no longer appears on the page.
- **BR-6 Delete** is a **soft delete** (`RecordStatus__c = 'deleted'`); all of the sprint's tickets have their `Sprint__c` cleared (returned to the backlog).
- **BR-7** Only sprints with status **not** `completed` and **not** `deleted` are loaded for the page.

5.2 Sprint story-point accounting

`TotalStoryPoint__c` = sum of the story points of all tickets in the sprint; `TotalEndedStoryPoint__c` = sum of story points of tickets currently in an **end status**; `storyPointsPercent` = round(ended / total × 100) (0 when total = 0). Maintained incrementally:

- **BR-8** Create ticket in sprint → `+SP` to total; `+SP` to ended only if the ticket's status is an end status.
- **BR-9** Move ticket backlog→sprint → `+SP` to total (and ended if end status).
- **BR-10** Move ticket sprint→backlog → `−SP` from total (and ended if end status).
- **BR-11** Delete ticket that is in a sprint → `−SP` from total (and ended if end status); bulk delete aggregates per sprint.Ti

- **BR-12** Change ticket status **to** an end status while in a sprint → `+SP` to ended only.
- **BR-13** Deleting a sprint detaches tickets without re-deriving totals (tickets simply lose their sprint link).

5.3 Ticket ordering & identity

- **BR-14** Tickets are ordered by `Score__c` (zero-padded 6-digit string so ASC string sort = ASC

integer sort). New inserts take `prevMax + gap (200)`. Backlog scope tracks the max on `Project__c.BacklogMaxScore__c`; sprint scope on `Sprint__c.MaxScore__c`.

- **BR-15** Reordering computes a score between neighbors; on collision the scope is **rebalanced**

(re-spread at gap intervals) in a single update.

- **BR-16** Moving a ticket between scopes reassigns its score to the destination scope's next score.
- **BR-17** A ticket cannot be moved into a sprint while it already belongs to a *different* sprint

(`requireTicketNotInDifferentSprint`).

- **BR-18 A Ticket Name must be unique per project** among non-deleted tickets; the name is

auto-generated (`NameSequenceService.nextTicketName`).

5.4 Ticket defaults & workflow

- **BR-19** On create, if Priority is blank it defaults to **Medium**; if State is blank it defaults

to the project's **start status**; `RecordStatus__c = 'active'`; `Creator__c` = the current user's active project member.

- **BR-20** A status change is allowed only if a `WorkflowTransition__c` exists for

`fromStatus → toStatus`; all `ValidateField__c` rules attached to that transition must pass before the change is applied.

- **BR-21** Reaching an end status returns `isEndStatus = true` so the UI can signal "no further

transitions".

5.5 Data hygiene

- **BR-22** All ticket/sub-task queries exclude soft-deleted rows (`RecordStatus__c != 'deleted'`

/ = 'active'), per the project's soql-exclude-deleted rule.

6. Use Case Descriptions

UC-1 — Load the Backlog page

- **Actor:** Project Member
- **Pre-conditions:** User is authenticated; a projectId exists in localStorage.
- **Main flow:**

1. Component mounts and reads projectId. 2. Calls loadBacklogData(projectId). 3. Renders the Backlog container and every future/in_progress sprint container (collapsed).

- **Alternative flows:**
- **A1 (no project):** No projectId → Choose-Project splash; on projectchosen the project is stored and the main flow resumes from step 2.
- **A2 (load error):** Apex returns success=false or throws → error message shown, content hidden.

UC-2 — Create a sprint

- **Actor:** Project Member
- **Pre-conditions:** Backlog page loaded for a project.
- **Main flow:**

1. User clicks **+ Sprint**. 2. Create-Sprint modal opens (Duration, Start Date, Goal). 3. User fills the form and clicks **Create**. 4. Client validation passes (VR-1..VR-3). 5. createSprint(duration, startDate, goal, projectId) runs; sprint saved as future. 6. New sprint container appears in the Sprints panel.

- **Alternative flows:**
- **A1 (validation fails):** Inline modal error; no Apex call.
- **A2 (duplicate name):** Server rejects via BR-2 → error toast.
- **A3 (cancel):** Modal closes, nothing saved.

UC-3 — Start a sprint

- **Actor:** Project Member
- **Pre-conditions:** A future sprint exists.
- **Main flow:**

1. User clicks **Start** on the sprint. 2. Confirmation "Start this sprint?" → confirm. 3. startSprint(sprintId) runs. 4. Status becomes in_progress; badge updates.

- **Alternative flows:**
- **A1 (another active sprint):** BR-4 violated → error toast; status unchanged.
- **A2 (cancel):** No call.

UC-4 — Complete a sprint

- **Actor:** Project Member
- **Pre-conditions:** Sprint is `in_progress` (and completable).
- **Main flow:**

1. User clicks **Complete** → confirm "Mark this sprint as complete?". 2. `completeSprint(sprintId)` runs; status `completed`. 3. Sprint container is removed from the page.

- **Alternative flows:**
- **A1 (not completable):** Error toast; sprint stays.
- **A2 (cancel):** No call.

UC-5 — Edit a sprint

- **Actor:** Project Member
- **Pre-conditions:** A non-deleted sprint exists.
- **Main flow:**

1. User clicks **Edit**; modal opens pre-filled with current duration/start date/goal. 2. User updates fields and clicks **Update**. 3. `updateSprint(sprintId, duration, startDate, goal)` runs. 4. Header dates and goal update in place (end date recomputed).

- **Alternative flows:**
- **A1 (validation fails):** Inline error; no call.
- **A2 (cancel):** Modal closes unchanged.

UC-6 — Delete a sprint

- **Actor:** Project Member
- **Pre-conditions:** A non-deleted sprint exists.
- **Main flow:**

1. User clicks **Delete** → confirmation pop-up "Delete this sprint? Tickets will be moved to backlog." → **Confirm**. 2. `deleteSprint(sprintId)` soft-deletes the sprint and clears `Sprint__c` on its tickets. 3. Sprint container disappears; its loaded tickets are appended to the backlog list.

- **Alternative flows:**

- **A1 (cancel):** Pop-up closes; nothing deleted.
- **A2 (server error):** Error toast; sprint stays.

UC-7 — Create a ticket (backlog or sprint)

- **Actor:** Project Member
- **Pre-conditions:** Backlog page loaded; at least one Ticket Type exists for the project.
- **Main flow:**

1. User clicks **+ Ticket** (backlog header) or **+ Ticket** (sprint header). 2. Reusable Create-Ticket modal opens (Summary, Description, Story Points, Ticket Type, State, Priority). 3. User fills it and clicks **Create**. 4. Client validation passes (VR-5..VR-10 as applicable). 5. `createTicketFromBacklog(...)` or `createTicketFromSprint(...)` runs (defaults applied per BR-19). 6. Ticket appears in the backlog list, or in the sprint with sprint totals updated (BR-8).

- **Alternative flows:**
- **A1 (validation fails):** Inline error; no call.
- **A2 (duplicate name):** Server rejects via BR-18 → error toast.
- **A3 (cancel):** Modal closes, nothing created.

UC-8 — Move a ticket between backlog and sprint

- **Actor:** Project Member
- **Pre-conditions:** Source and target containers visible; sprint expanded for sprint-side moves.
- **Main flow (backlog→sprint):**

1. User drags a backlog ticket onto a sprint container and drops. 2. `moveTicketToSprint(ticketId, sprintId)` links the ticket and increases sprint totals (BR-9). 3. Ticket leaves the backlog list and appears under the sprint; the story-point line updates.

- **Main flow (sprint→backlog):**

1. User drags a sprint ticket onto the backlog and drops. 2. `moveTicketToBacklog(ticketId)` clears the sprint, decreases totals (BR-10), re-scores the ticket. 3. Ticket leaves the sprint and appears in the backlog.

- **Alternative flows:**
- **A1 (sprint→different sprint):** Drop is disallowed (no cross-sprint move); nothing happens.
- **A2 (ticket already in a different sprint):** BR-17 rejects → error toast.

UC-9 — Reorder a ticket within a container

- **Actor:** Project Member
- **Pre-conditions:** Container has ≥ 2 tickets.
- **Main flow:**

1. User drags a ticket over another ticket (or the top drop-zone) in the **same** container and drops. 2. `moveTicketPosition(movedTicketId, beforeTicketId)` recomputes the moved ticket's `Score__c` (BR-15). 3. The list re-renders in the new order.

- **Alternative flows:**
- **A1 (different container):** Drop ignored by the reorder handler (falls through to move handlers).
- **A2 (dropped on itself):** No-op.

UC-10 — Operate on a ticket inline

- **Actor:** Project Member
- **Pre-conditions:** Ticket row visible.
- **Main flow (any one of):**
- Edit summary → `updateTicketSummary`.
- Change status (combo) → `changeTicketState` (BR-20); end status raises the "no further transitions" toast and may bump sprint ended SP (BR-12).
- Change priority (combo → ✓) → validate (VR-10) → `updateTicketPriority`.
- Change assignee (combo) → `assignTicket`.
- Set/clear epic (epic label → modal) → `updateTicketEpic`, or `createEpic` then assign.
- Delete → `deleteTicket` (sprint totals adjusted per BR-11).
- Add sub-task (+) → `createSubtask`; Expand → `loadSubtasks`.
- **Alternative flows:**
- **A1 (priority invalid/empty):** VR-10 error toast; no call.
- **A2 (transition not allowed / validate-field fails):** BR-20 error toast; the combo is re-keyed to revert.
- **A3 (server error on any action):** Error toast; optimistic UI reverts where applicable.

UC-11 — Bulk delete tickets

- **Actor:** Project Member
- **Pre-conditions:** ≥ 1 ticket selected.
- **Main flow:**

1. User selects tickets; bulk bar shows the count. 2. User clicks **Delete Selected** → confirm "Delete N ticket(s)?". 3. `deleteTickets(ticketIds)` soft-deletes them; affected sprint totals recomputed (BR-11). 4. Rows are removed; selection cleared.

- **Alternative flows:**
- **A1 (cancel):** Selection cleared via **Cancel**, nothing deleted.

UC-12 — Create / expand sub-tasks

- **Actor:** Project Member
- **Pre-conditions:** Ticket row visible.
- **Main flow:**

1. User clicks **+** on a ticket → Create-Subtask modal (Summary required). 2. `createSubtask(...)` runs; sub-task added under the ticket. 3. User clicks the ticket's expand chevron → `loadSubtasks` lists all sub-tasks.

- **Alternative flows:**
- **A1 (summary blank):** Required-field validation; no call.
- **A2 (cancel):** Modal closes.

UC-13 — Operate on a ticket from the ticket view (peek panel)

- **Actor: Project Member**
- **Pre-conditions: A ticket row is visible on the Backlog page.**
- **Main flow (any one of):**
- **Open → click a ticket row → ticketviewopen renders c-ticket-view for that ticket.**
- **Edit summary → confirm → ticketsummaryupdate → updateTicketSummary.**
- **Change status (combo) → ticketstatuschange → changeTicketState (BR-20).**
- **Edit description → click the description → rich-text editor → Save → ticketdescriptionupdate → updateTicketDescription.**
- **Expand linked items → ticketlinkedtoexpand → loadTicketLinkedTo lists them; + → choose a link type and search a ticket (≥ 2 chars, ticketsearch → loadTicketBySearchTerm) → Link → linkToTicket.**
- **Expand sub-tasks → subtasksexpand → loadSubtasks lists them; + → fill the create-subtask form → Create → createSubtask.**
- **Close → closeticketview hides the panel.**
- **Alternative flows:**
- **A1 (summary blank): client validation blocks the save; no call.**

- **A2 (link incomplete): Link stays disabled until both a type and a target ticket are chosen; ticket search ignores terms shorter than 2 characters.**
- **A3 (server error on any action): error toast; the panel keeps the last good value.**

7. Non-Functional Notes

- **Pagination / governor safety:** page size 5 everywhere; backlog and sprint reads are `cacheable=true`; sprint tickets load lazily on expand. Bulk delete and sprint-total updates are written to be bulk-safe (one query/DML per object).
- **Derived UI state:** the LWC keeps one principal state (`sprints`, `backlogTickets`) and derives every displayed value via getters / formatter utilities (`formatSprint`, `enrichTickets`).
- **Soft delete everywhere:** sprints, tickets, and sub-tasks are soft-deleted; queries filter them out.